DOCUMENTACIÓN DE PROPUESTA TÉCNICA DEL SOFTWARE

PROYECTO: Traduce Señas

INTEGRANTES:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Análisis y Desarrollo De Software

2026

Tabla de contenido

Introducción 3
Descripción del problema y de la necesidad 3
Solución propuesta 3
Alcance técnico 4
Tecnologías seleccionadas 5
Alternativas evaluadas 5
Arquitectura propuesta 6
Metodología de desarrollo 6
Cronograma 7
Recursos 7
    Talento humano 7
    Recursos técnicos 7
Presupuesto 8
Riesgos técnicos principales 8
Entregables 8
Conclusiones 9
Referencias 10

Introducción

La propuesta técnica describe la solución de software que el equipo plantea para el problema de comunicación entre personas sordas y oyentes, junto con las tecnologías elegidas, la arquitectura, la metodología de trabajo, el cronograma, los recursos y el presupuesto necesarios para construirla.

El documento está dirigido al instructor y a cualquier evaluador técnico que necesite entender por qué la solución está construida como está y qué alternativas se descartaron.

Descripción del problema y de la necesidad

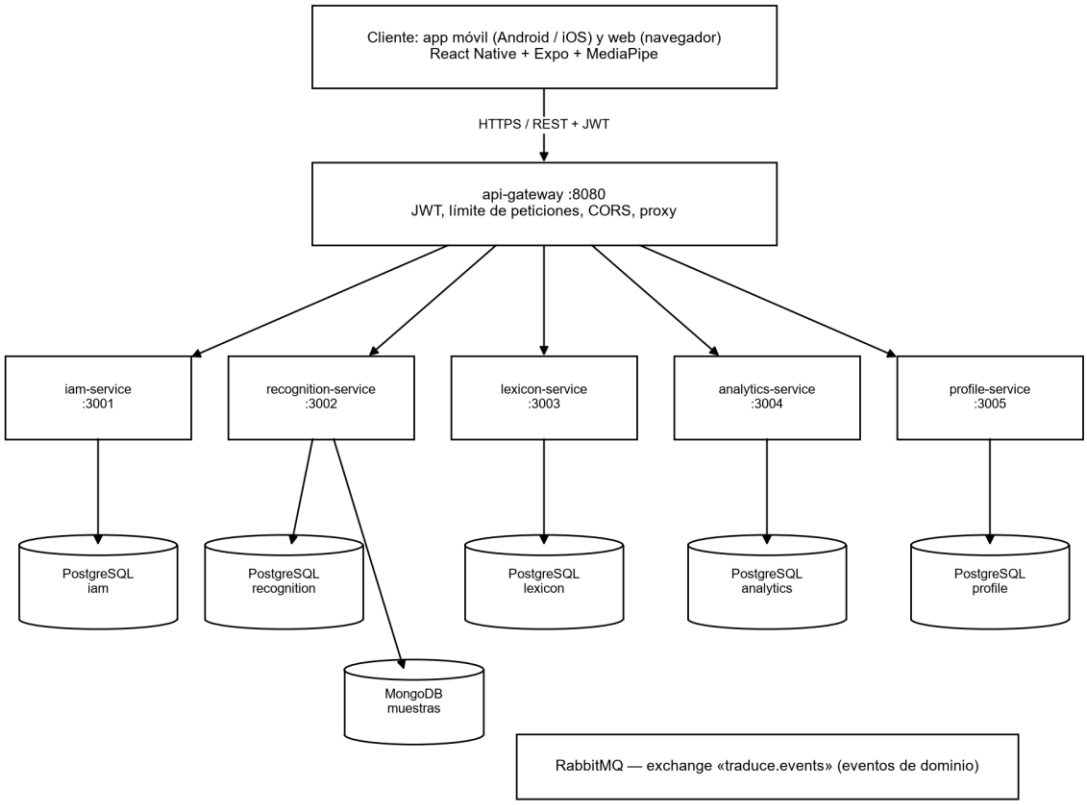
En Colombia la cantidad de intérpretes de LSC es muy inferior a la de personas sordas que los necesitan (INSOR, 2020). La Organización Mundial de la Salud (2021) estima que más de 1 500 millones de personas en el mundo viven con algún grado de pérdida auditiva. Sin un intérprete, una persona sorda no puede comunicarse con fluidez en el colegio, la tienda o la ventanilla de una entidad pública. Se necesita una herramienta disponible en cualquier teléfono que traduzca las señas a texto o voz de forma inmediata.

Solución propuesta

Se propone una aplicación multiplataforma que usa la cámara del dispositivo para reconocer las señas y mostrar su traducción. La solución se compone de:

- **Cliente multiplataforma** en React Native con Expo, que se compila para Android, iOS y web (escritorio) a partir de una sola base de código.
- **Motor de reconocimiento en el dispositivo** con MediaPipe Hands (Zhang et al., 2020), que extrae 21 puntos de la mano por cuadro, y un clasificador por plantillas basado en Dynamic Time Warping.
- **Backend de microservicios** en Node.js, Express y TypeScript, detrás de un API Gateway, con una base de datos por servicio.
- **Comunicación por eventos** con RabbitMQ para mantener los servicios desacoplados.
- **Despliegue con contenedores** Docker y Docker Compose, reproducible en cualquier servidor.

Figura 1
Vista general de la arquitectura propuesta



Nota. Elaboración propia.

Alcance técnico

Tabla 1
Módulos funcionales y servicio responsable

Módulo	Funcionalidad	Servicio
Gestión de usuarios	Registro, verificación de correo, inicio de sesión, recuperación de contraseña, roles	iam-service
Traducción	Reconocimiento de señas, texto y voz, historial	App + recognition-service
Entrenamiento	Captura de muestras, plantillas de señas, registro de modelos	recognition-service
Léxico y alfabeto	Catálogo de señas, categorías, alfabeto con modelos 3D	lexicon-service
Perfil	Datos personales, tema, idioma, notificaciones y logros	profile-service
Estadísticas	Eventos de uso y tableros para el administrador	analytics-service
Entrada única	Enrutamiento, validación de JWT, límite de peticiones, CORS	api-gateway

Tecnologías seleccionadas

Tabla 2
Pila tecnológica y justificación

Capa	Tecnología	Versión	Justificación
Cliente	React Native + Expo	RN 0.86 / Expo 57	Una sola base de código para Android, iOS y web; el equipo domina JavaScript
Cliente	React	19.2	Biblioteca de interfaz común a móvil y web
Cliente	React Navigation	6.x	Navegación por pilas y pestañas estándar en React Native
Visión	MediaPipe Tasks Vision	0.10.14	Detección de mano preentrenada y en tiempo real en el dispositivo
Visión	TensorFlow.js	4.22	Ejecución de modelos de clasificación en el cliente
Voz	expo-speech	57.x	Síntesis de voz nativa para la salida en audio
Backend	Node.js + Express	20 LTS / 5.2	Mismo lenguaje que el cliente; suficiente para el tráfico del proyecto
Backend	TypeScript	5.6+	Tipado estático que reduce errores en una base de código compartida
Datos	PostgreSQL	16	Base relacional robusta y gratuita, una por servicio
Datos	MongoDB	7	Documentos flexibles para las muestras de entrenamiento
Mensajería	RabbitMQ	3.x	Eventos entre servicios; sencillo y con panel web
Seguridad	JWT (HS256) + bcrypt	—	Autenticación sin estado y contraseñas con hash
Infraestructura	Docker + Docker Compose	24+ / 2.20+	Entorno reproducible con un solo comando
Control de versiones	Git + GitHub	—	Ramas por funcionalidad, pull requests y revisión

Alternativas evaluadas

Tabla 3
Decisiones técnicas y alternativas descartadas

Decisión	Alternativa descartada	Motivo
Microservicios con API Gateway	Monolito modular	Permite escalar el reconocimiento por separado y repartir el trabajo por dominio; decisión confirmada por el líder del producto el 4 de octubre de 2026
PostgreSQL por servicio	MySQL compartido	Aislamiento de datos por servicio, tipos JSONB y restricciones más expresivas
RabbitMQ	Apache Kafka	Kafka es desproporcionado para el volumen del proyecto; RabbitMQ es más simple de operar
Reconocimiento en el dispositivo	Envío de video al servidor	Menor latencia, menor consumo de datos y el video no sale del teléfono (privacidad)
Plantillas + DTW	Red neuronal entrenada desde cero	Funciona con pocos ejemplos por seña y es explicable; la red queda como mejora futura

Decisión	Alternativa descartada	Motivo
Expo (React Native)	Desarrollo nativo separado (Kotlin y Swift)	Un equipo de tres personas no puede mantener tres bases de código

Arquitectura propuesta

Cada microservicio sigue la arquitectura hexagonal (Cockburn, 2005): el dominio y los casos de uso no dependen de Express ni de la base de datos, sino de puertos que implementan los adaptadores. Esto facilita las pruebas y permite cambiar la tecnología de persistencia sin tocar la lógica de negocio.

El cliente se organiza por funcionalidades (feature/) con capas de presentación, negocio, datos, servicios externos y configuración. El detalle se encuentra en el documento de diseño del software.

Metodología de desarrollo

El proyecto se gestiona con Scrum (Schwaber y Sutherland, 2020) en sprints de dos semanas. El instructor actúa como dueño del producto que revisa y acepta cada incremento, y el líder técnico cumple también el rol de Scrum Master.

Tabla 4
Eventos y artefactos de Scrum en el proyecto

Elemento	Aplicación en el proyecto
Sprint	Dos semanas; 14 sprints planificados entre el 1 de enero y el 31 de julio de 2026
Planeación	Selección de historias de usuario del backlog según prioridad MoSCoW
Revisión	Demostración del incremento al instructor
Retrospectiva	Acciones de mejora registradas en el documento de acciones correctivas
Product backlog	Historias de usuario trazadas a los requisitos (03-product/product-backlog.md)
Definición de hecho	Código revisado, pruebas en verde, accesibilidad verificada y documentación actualizada
Flujo de Git	Rama por funcionalidad, commits en inglés con Conventional Commits, pull request con squash and merge

Cronograma

Tabla 5
Cronograma por fases

Fase	Sprints	Periodo	Entregables
Análisis y requisitos	1–2	enero de 2026	Informe de requisitos, historias de usuario, casos de uso
Diseño	3–4	febrero de 2026	Arquitectura, modelo de datos, prototipos de interfaz
Construcción del núcleo	5–9	marzo a mayo de 2026	Autenticación, traducción, alfabeto, historial
Módulos complementarios	10–12	junio de 2026	Perfil, notificaciones, estadísticas, logros
Pruebas e implantación	13–14	julio de 2026	Plan y resultados de pruebas, despliegue, manuales
Reestructuración a microservicios	Re-planeación	septiembre a octubre de 2026	Separación del backend en seis componentes y documentación actualizada

Nota. La línea base original cerró el 31 de julio de 2026; la fase de microservicios corresponde a la re-planeación posterior.

Recursos

Talento humano

Tabla 6
Equipo del proyecto

Nombre	Rol	Responsabilidad	Contacto
Juan Mauricio Suaza Solórzano	Líder técnico, Scrum Master, analista y desarrollador	Análisis, diseño, programación del backend (lexicon y recognition) y coordinación	suazasolorzano@gmail.com
Luis Alejandro Duarte Aldana	Analista y desarrollador	Análisis, diseño de interfaces y programación	aleosea777@gmail.com
Juan Camilo Fierro Esquivel	Analista y desarrollador	Análisis, documentación de producto y programación	fierroesquiveljuancamilo@gmail.com

Nota. Aprendices del programa Tecnólogo en Análisis y Desarrollo de Software del SENA.

Recursos técnicos

- Tres computadores de desarrollo con al menos 16 GB de RAM y Docker Desktop.
- Teléfonos Android e iOS de gama media para pruebas de cámara y rendimiento.
- Servidor (físico o en la nube) con 4 vCPU, 8 GB de RAM y 50 GB de disco para el despliegue.
- Cuentas de GitHub, servidor de correo SMTP y cuentas de desarrollador de Google y

Facebook para el inicio de sesión externo.

Presupuesto

Tabla 7
Presupuesto estimado del proyecto (pesos colombianos)

Rubro	Detalle	Valor (COP)
Desarrollo y programación	Tres desarrolladores durante seis meses (líder: $3 000 000/mes; analistas: $2 500 000/mes cada uno)	48 000 000
Hardware	Equipos de cómputo y dispositivos móviles de prueba	15 003 310
Infraestructura y servidores	Servidor de despliegue y dominio	1 350 000
Pruebas y capacitación	Pruebas piloto y capacitación de usuarios	2 000 000
Software y seguridad	Certificados y herramientas	225 000
Total		**66 578 310**

Nota. El total corresponde a la suma de los componentes de la Propuesta Técnica y Económica; es la cifra que se toma como válida mientras se concilian los totales parciales que circulan en otros documentos.

Riesgos técnicos principales

Tabla 8
Riesgos y estrategia de mitigación

Riesgo	Nivel	Mitigación
Datos de entrenamiento de LSC insuficientes	Crítico	Usar el conjunto público LSC-54 y capturar muestras con consentimiento; reentrenar con las señas más fallidas
Precisión del reconocimiento menor a la esperada	Alto	Umbral que evita mostrar traducciones dudosas; métricas por seña antes de publicar plantillas
Equipo pequeño frente a la complejidad de los microservicios	Alto	Paquete compartido (@traduce/shared), plantillas de servicio y Docker Compose
Privacidad de las muestras (Ley 1581 de 2012)	Alto	Consentimiento obligatorio por muestra, acceso solo para ADMIN, video procesado en el dispositivo
Servidor único sin redundancia	Medio	Copias de seguridad diarias y procedimiento de restauración documentado

Entregables

1. Informe de especificación de requisitos.
2. Documentación de análisis (vistas dinámicas).
3. Propuesta técnica (este documento).
4. Documentación de diseño del software.
5. Artefactos de código: aplicación móvil, web y de escritorio (vía navegador) y backend.
6. Documentación de pruebas.

7. Procedimiento de implementación.
8. Documentación de acciones correctivas, preventivas y de mejoramiento.
9. Manuales técnico, de instalación y de usuario final.
10. Artículo del proyecto.

Conclusiones

La solución propuesta es viable para un equipo de tres aprendices porque se apoya en componentes maduros (MediaPipe, Expo, PostgreSQL, RabbitMQ) y concentra el esfuerzo propio en el reconocimiento de la LSC y en la experiencia accesible. La arquitectura de microservicios permite crecer por partes y la ejecución del reconocimiento en el dispositivo cumple el requisito de tiempo real sin exponer el video del usuario.

Referencias

Broadcom. (2026). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Docker Inc. (2026). *Docker Compose documentation*. https://docs.docker.com/compose/

Expo. (2026). *Expo documentation*. https://docs.expo.dev/

Instituto Nacional para Sordos. (2020). *Observatorio social de la población sorda colombiana*. INSOR. https://www.insor.gov.co/observatorio/

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines* (arXiv:1906.08172). arXiv. https://doi.org/10.48550/arXiv.1906.08172

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Organización Mundial de la Salud. (2021). *Informe mundial sobre la audición*. https://www.who.int/publications/i/item/9789240020481

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking* (arXiv:2006.10214). arXiv. https://doi.org/10.48550/arXiv.2006.10214